Travelpedia — Technical overview (for new web developers)

Audience: Beginners in web development who want to understand how this project is built and where each piece lives.

What this app does (in one sentence): Travelpedia is a tour marketplace where **operators** list trips, **visitors** browse active listings and send inquiries, and **admins** approve operators and tours before they go live.

---

1. The technology stack

Piece	What it is	Role in Travelpedia
Next.js	A React framework	Runs the website, routes (URLs), and server-side logic
React	A UI library	Builds buttons, forms, and pages in the browser
TypeScript	JavaScript with types	Used across the project for safer code
Tailwind CSS	Utility-first CSS	Styling (colors, spacing, layout)
Supabase	Backend-as-a-service	**Database**, **authentication**, and **security rules** in one cloud product

There is **no separate custom server** (like Express or Django) in this repo: **Supabase** provides the database and auth API, and **Next.js** talks to it using the Supabase client libraries.

---

2. Where is the database?

The database is **not** a file inside your project folder. It lives in the **cloud**, on your **Supabase project**.

- **Engine:** PostgreSQL (a relational database).
- **You manage it** through the Supabase Dashboard: tables, SQL, authentication users, and email settings.
- **The app connects** using two values in `.env.local` (see `.env.example` for names):
  - `NEXT_PUBLIC_SUPABASE_URL` — your project's API URL
  - `NEXT_PUBLIC_SUPABASE_ANON_KEY` — a **public** key used by the browser and server; real security is enforced by **Row Level Security (RLS)** in the database, not by hiding this key.

Main tables (conceptual)

Defined in `supabase/migrations/001_initial_schema.sql`:

1. **`profiles`** — One row per signed-up user. Links to Supabase Auth (`auth.users`) by the same UUID. Stores role (`operator` or `admin`), approval status, company name, etc.
2. **`tours`** — Tour listings (destination, dates, price, status `pending` / `active` / `closed`, etc.).
3. **`inquiries`** — Messages from visitors about a tour (name, phone, seats, message).

Row Level Security (RLS): PostgreSQL policies decide who can **read**, **insert**, or **update** rows. For example, the public site may only **read** tours where `status = 'active'`. Operators can manage their own tours only if they are **approved**. Admins get broader access via helper functions like `is_admin()` in SQL.

---

3. Frontend vs backend — how this project splits them

Big picture

- **Frontend** = what runs in the **user's browser**: React components, forms, clicks, and the Supabase **browser** client (for login/register).
- **Backend** = logic that runs on the **server** (Next.js server): reading cookies, checking the logged-in user, fetching data before sending HTML, and **Server Actions** that mutate data safely.

In Next.js App Router terms

- **Server Components** (default): Files like `layout.tsx` and many `page.tsx` files run on the server. They can use `createClient()` from `@/lib/supabase/server` and `await` database calls. Users never see your database password here; only the anon key and RLS apply.
- **Client Components** (`"use client"`): Used when you need browser interactivity (forms, `useState` , `useRouter`). Examples: `login-form.tsx` , `register-form.tsx` . They use `createClient()` from `@/lib/supabase/client` .
- **Server Actions** (`"use server"`): Functions in files like `src/app/actions/admin.ts` that only run on the server. They verify the user is an admin, then update Supabase (e.g. approve a tour).

Request flow (simplified)

1. User opens a URL → Next.js renders a route (e.g. `/tours`).
2. For public tour listings, the server may call `fetchActiveTours()` in `src/lib/tours-query.ts` , which uses an **anonymous** Supabase client (no login cookie) to read only **active** tours — allowed by RLS.
3. For `/admin` or `/operator` , a **layout** checks `supabase.auth.getUser()` and the `profiles` row, then **redirects** if the user is not allowed.

Middleware

`src/middleware.ts` refreshes the Supabase **session** (auth cookies) on each request so login state stays in sync. Route protection for `/admin` and `/operator` is mainly done in **layouts**, not only in middleware.

---

4. How to access the admin account

Important: The registration form **always** creates an **operator** profile with `approval_status = pending` . The app **does not** let users register as admin through the UI. That is intentional for security.

From `supabase/migrations/001_initial_schema.sql` :

- A database trigger `on_auth_user_created` runs when a new row is added to `auth.users` and inserts a `profiles` row with `role = 'operator'` and `approval_status = 'pending'` .

Promoting a user to admin

1. **Create a user** in Supabase Dashboard: **Authentication** → **Users** → **Add user** (or register via `/auth/register` and confirm email if required).
2. In Supabase: **SQL Editor**, run (replace the UUID with that user's id from Authentication → Users):

```sql
update public.profiles
set role = 'admin', approval_status = 'approved'
where id = '<auth-user-uuid>';
```

3. Sign in at `/auth/login` with that user's email and password. The login form sends **admins** to `/admin` and **operators** to the `next` query param or operator dashboard.

Who can open `/admin`

`src/app/admin/layout.tsx` requires:

- A valid logged-in user, and
- `profiles.role === 'admin'` **and** `profiles.approval_status === 'approved'`.

Otherwise the user is redirected to login.

---

5. How email verification works

Email confirmation is handled by **Supabase Auth**, not by custom code in this repo.

What the app does

In `src/app/auth/register/register-form.tsx`, sign-up calls:

```
supabase.auth.signUp({ email, password, options: { data: { ... } } })
```

After a successful response, the user sees: *"Check your email to confirm your account, then sign in."*

What Supabase does

- If **"Confirm email"** is enabled in Supabase (**Authentication** → **Providers** → **Email**), Supabase sends a confirmation link to the user's inbox.
- Until the user clicks that link, they may not be able to sign in (depending on your project settings).
- You can adjust behavior in the dashboard: disable confirmation for local development, change redirect URLs, or use Supabase's **email templates**.

Local development

For testing without real email, developers often use Supabase's dashboard to confirm users manually or temporarily turn off email confirmation in project settings. Always re-enable confirmation for production.

---

6. Operator approval vs tour approval

- **Operators:** After registration, `approval_status` is `pending` until an admin approves them in `/admin/operators` (via Server Actions in `src/app/actions/admin.ts`). Until approved, operators cannot publish tours (enforced by RLS `is_approved_operator()`).
- **Tours:** New tours start as `pending`. Admins approve them to `active` so they appear on the public site; RLS only exposes `active` tours to anonymous readers.

---

7. Running the project locally

1. Install dependencies: `npm install`
2. Copy `.env.example` to `.env.local` and set your Supabase URL and anon key from your Supabase project settings.
3. Apply the SQL in `supabase/migrations/001_initial_schema.sql` in the Supabase SQL Editor (if not already applied).

4. Run: `npm run dev` and open the URL shown (usually `http://localhost:3000`).

---

8. File map (quick reference)

Area	Typical files
Pages & layouts	`src/app/**/page.tsx`, `layout.tsx`
Auth UI	`src/app/auth/login/`, `src/app/auth/register/`
Admin actions	`src/app/actions/admin.ts`
Operator tour actions	`src/app/actions/operator-tours.ts`
Public tour queries	`src/lib/tours-query.ts`
Supabase clients	`src/lib/supabase/client.ts`, `server.ts`, `middleware.ts`
DB schema & RLS	`supabase/migrations/001_initial_schema.sql`
TypeScript types	`src/types/database.ts`

---

9. Security notes for beginners

- The **anon key** is safe to expose in the frontend **only** because **RLS** defines what each user (or anonymous visitor) can do. Never disable RLS without understanding the impact.
- **Service role** keys (if you ever add them) must stay **server-only** and never ship to the browser — this project currently relies on the anon key plus RLS for app access.
- Keep `.env.local` out of git (it is usually gitignored); use `.env.example` only as a template without real secrets.

Generated for the Travelpedia codebase. Stack versions and paths refer to the repository at the time this document was written.